backend

controllers/auth.controller / beginAuthentication /

Function beginAuthentication

```
beginAuthentication(
    req: Request<unknown, BeginAuthenticationBody>,
    res: Response<PublicKeyCredentialRequestOptionsJSON>,
    next: NextFunction,
): Promise<void>
```

Begin WebAuthn authentication by generating authentication options.

Parameters

- req: *Request<unknown, BeginAuthenticationBody>*
  - **body.email:**

    Email of the user attempting to authenticate
- res: *Response<PublicKeyCredentialRequestOptionsJSON>*
- next: *NextFunction*

Returns *Promise<void>*

JSON containing WebAuthn authentication options

Remarks

Route POST /api/auth/create-authentication-options

This endpoint returns the challenge and parameters needed for the browser to begin WebAuthn authentication.

Src: https://simplewebauthn.dev/docs/packages/server#1-generate-authentication-options

Route POST /api/auth/create-authentication-options Route POST /api/auth/create-authentication-options

Throws

AppError if authentication options cannot be generated

Defined in controllers/auth.controller.ts:218

Generated using TypeDoc

controllers/auth.controller / beginRegistration /

Function beginRegistration

```
beginRegistration(
    req: Request<unknown, BeginRegistrationBody>,
    res: Response<PublicKeyCredentialCreationOptionsJSON>,
    next: NextFunction,
): Promise<void>
```

Begin WebAuthn registration by generating registration options.

Parameters

- req: *Request<unknown, BeginRegistrationBody>*
  - **body.email:**

    Email of the user attempting to register
- res: *Response<PublicKeyCredentialCreationOptionsJSON>*
- next: *NextFunction*

Returns *Promise<void>*

JSON containing WebAuthn registration options

Remarks

Route POST /api/auth/create-registration-options

This endpoint returns the WebAuthn registration challenge and parameters

Src: https://simplewebauthn.dev/docs/packages/server#1-generate-registration-options

Throws

AppError if registration options cannot be generated

Defined in controllers/auth.controller.ts:64

backend

controllers/auth.controller / finishAuthentication /

Function finishAuthentication

```
finishAuthentication(
    req: Request<unknown, FinishAuthenticationBody>,
    res: Response<FinishAuthenticationResponse>,
    next: NextFunction,
): Promise<
    Response<FinishAuthenticationResponse, Record<string, any>>
    | undefined,
>
```

Complete WebAuthn authentication by verifying the assertion response.

Parameters

- req: *Request<unknown, FinishAuthenticationBody>*
  - **body.email:**

    User email
  - **body.assertionResponse:**

    WebAuthn assertion response
- res: *Response<FinishAuthenticationResponse>*
- next: *NextFunction*

Returns *Promise<Response<FinishAuthenticationResponse, Record<string, any>> | undefined>*

JSON containing verification result and JWT token if successfully verified

Remarks

Route POST /api/auth/finish-authentication

If verification succeeds, a JWT token is returned for session management.

Src: https://simplewebauthn.dev/docs/packages/server#2-verify-authentication-response

Throws

AppError if verification fails or user cannot be found

Defined in controllers/auth.controller.ts:252

Generated using TypeDoc

backend

controllers/auth.controller / finishRegistration /

Function finishRegistration

```
finishRegistration(
    req: Request<unknown, FinishRegistrationBody>,
    res: Response<FinishRegistrationResponse>,
    next: NextFunction,
): Promise<
    Response<FinishRegistrationResponse, Record<string, any>>
    | undefined,
>
```

Complete WebAuthn registration by verifying the attestation response.

Parameters

- req: *Request<unknown, FinishRegistrationBody>*
  - **body.email:**

    Email of the registering user
  - **body.attestationResponse:**

    WebAuthn attestation response
- res: *Response<FinishRegistrationResponse>*
- next: *NextFunction*

Returns *Promise<Response<FinishRegistrationResponse, Record<string, any>> | undefined>*

JSON containing verification result and JWT token if successfully verified

Remarks

Route POST /api/auth/finish-registration

After the browser creates a credential, it sends the attestation response back to this endpoint. If verification succeeds, the users account is created and a JWT token is returned.

Src: https://simplewebauthn.dev/docs/packages/server#2-verify-registration-response

Throws

AppError if verification fails or user cannot be created

Defined in controllers/auth.controller.ts:100

Generated using TypeDoc

backend

controllers/auth.controller / loginPassword /

Function loginPassword

```
loginPassword(
    req: Request<unknown, PasswordLoginBody>,
    res: Response<PasswordAuthenticationResponse>,
    next: NextFunction,
): Promise<
    | Response<PasswordAuthenticationResponse, Record<string, any>>
    | undefined,
>
```

Authenticate using email & password.

Parameters

- req: *Request<unknown, PasswordLoginBody>*
  - **body.email:**

    Users email.
  - **body.password:**

    Users plain password
- res: *Response<PasswordAuthenticationResponse>*
- next: *NextFunction*

Returns *Promise<*
| Response<PasswordAuthenticationResponse, Record<string, any>>
| undefined,
>

JSON containing authentication status, token and user info

Remarks

Route POST /api/auth/login-password

Throws

AppError if credentials do not match or are missing.

Defined in controllers/auth.controller.ts:176

Generated using TypeDoc

controllers/auth.controller / registerPassword /

Function registerPassword

```
registerPassword(
    req: Request<unknown, PasswordLoginBody>,
    res: Response<PasswordAuthenticationResponse>,
    next: NextFunction,
): Promise<
    | Response<PasswordAuthenticationResponse, Record<string, any>>
    | undefined,
>
```

Register a user using email & password.

Parameters

- req: *Request<unknown, PasswordLoginBody>*
  - **body.email:**

    User email
  - **body.password:**

    User password
- res: *Response<PasswordAuthenticationResponse>*
- next: *NextFunction*

Returns *Promise<*
| Response<PasswordAuthenticationResponse, Record<string, any>>
| undefined,
>

JSON containing authentication status, token, and user info

Remarks

Route POST /api/auth/register-password

This is included as backup or alternative to WebAuthn registration as the functionality is still rather new.

Throws

AppError if email/password missing or user creation fails

Defined in controllers/auth.controller.ts:139

Generated using TypeDoc

Function helloThere

```
helloThere(req: AuthenticatedRequest, res: Response): Promise<void>
```

Test endpoint to verify that authentication is functioning correctly.

Parameters

- req: *AuthenticatedRequest*
- res: *Response*

Returns *Promise<void>*

A simple JSON message confirming access to a protected route

Remarks

Route GET /api/protected/hello-there

This route is protected and requires a valid JWT.

Defined in controllers/protected.controller.ts:23

backend

controllers/protected.controller / addRestaurantInfo /

Function addRestaurantInfo

```
addRestaurantInfo(
    req: AuthenticatedRequest,
    res: Response,
    next: NextFunction,
): Promise<void>
```

For adding restaurant information manually to the database.

Parameters

- req: *AuthenticatedRequest*
- res: *Response*
- next: *NextFunction*

Returns *Promise<void>*

A success message once the restaurant has been added

Remarks

Route POST /api/protected/add-restaurant-information

This route is protected and requires a valid JWT.

The request body must contain a valid restaurant data object. The data is validated and then inserted into the database.

Throws

AppError if `restaurantData` is missing or invalid

Throws

AppError if database insertion fails

Defined in controllers/protected.controller.ts:115

Generated using TypeDoc

controllers/protected.controller / parseFromSite /

Function parseFromSite

```
parseFromSite(
    req: AuthenticatedRequest,
    res: Response,
    next: NextFunction,
): Promise<void>
```

Parses restaurant data from an external website and stores it in the database.

Parameters

- req: *AuthenticatedRequest*
- res: *Response*
- next: *NextFunction*

Returns *Promise<void>*

A success message once the restaurant has been parsed and stored

Remarks

Route POST /api/protected/parse-from-site

This route is protected and requires a valid JWT.

The provided URL is forwarded to a microservice that extracts restaurant information. The parsed data is then validated and inserted into the database.

Throws

AppError if `restaurantUrl` is missing or invalid

Throws

AppError if parsing or database insertion fails

Defined in controllers/protected.controller.ts:77

Generated using TypeDoc

controllers/protected.controller / previewFromSite /

Function previewFromSite

```
previewFromSite(
    req: AuthenticatedRequest,
    res: Response,
    next: NextFunction,
): Promise<void>
```

Parses restaurant data from an external website and returns it for preview before saving it to a database.

Parameters

- req: *AuthenticatedRequest*
- res: *Response*
- next: *NextFunction*

Returns *Promise<void>*

The parsed restaurant data.

Remarks

Route POST /api/protected/preview-from-site

This route is protected and requires a valid JWT.

The provided URL is forwarded to a microservice that extracts restaurant information. The data is then returned as is to the frontend.

Throws

AppError if `restaurantUrl` is missing or invalid

Throws

AppError if parsing fails

Defined in controllers/protected.controller.ts:46

Generated using TypeDoc

backend

controllers/restaurant.controller / getRestaurantAndMenuData /

Function getRestaurantAndMenuData

```
getRestaurantAndMenuData(
    req: Request,
    res: Response<AllRestaurantData[]>,
    next: NextFunction,
): Promise<Response<AllRestaurantData[], Record<string, any>> | undefined>
```

Get all restaurant data with menus.

Parameters

- req: *Request*
- res: *Response*<*AllRestaurantData*[]>
- next: *NextFunction*

Returns *Promise*<*Response*<*AllRestaurantData*[], *Record*<*string*, *any*>> | *undefined*>

JSON array of restaurants with menu items

Remarks

Route: GET /api/restaurant/get-restaurant-data

This endpoint returns a list of all restaurants stored in the database, each including its associated menu items (if any exists).

Throws

AppError if data query fails

Defined in controllers/restaurant.controller.ts:18

Generated using TypeDoc

Type Alias BeginAuthenticationBody

```
type BeginAuthenticationBody = {
    email: string;
}
```

Defined in controllers/types.ts:35

Index

Properties

Ⓟ email

Properties

email

```
email: string
```

Defined in controllers/types.ts:36

Type Alias BeginRegistrationBody

```
type BeginRegistrationBody = {
    email: string;
}
```

Defined in controllers/types.ts:26

Index

Properties

Ⓟ email

Properties

email

```
email: string
```

Defined in controllers/types.ts:27

Type Alias FinishAuthenticationBody

```
type FinishAuthenticationBody = {
    assertionResponse: AuthenticationResponseJSON;
    email: string;
}
```

Defined in controllers/types.ts:39

Index

Properties

- P assertionResponse
- P email

Properties

assertionResponse

```
assertionResponse: AuthenticationResponseJSON
```

Defined in controllers/types.ts:41

email

```
email: string
```

Defined in controllers/types.ts:40

Generated using TypeDoc

Type Alias FinishAuthenticationResponse

```
FinishAuthenticationResponse: VerifiedAuthenticationResponse & {
    token?: string;
}
```

Defined in controllers/types.ts:14

backend

controllers/types / FinishRegistrationBody /

Type Alias FinishRegistrationBody

```
type FinishRegistrationBody = {
    attestationResponse: RegistrationResponseJSON;
    email: string;
}
```

Defined in controllers/types.ts:30

Index

Properties

- (P) attestationResponse
- (P) email

Properties

attestationResponse

```
attestationResponse: RegistrationResponseJSON
```

Defined in controllers/types.ts:32

email

```
email: string
```

Defined in controllers/types.ts:31

Generated using TypeDoc

Type Alias FinishRegistrationResponse

```
FinishRegistrationResponse: VerifiedRegistrationResponse & { token?: string }
```

Defined in controllers/types.ts:10

Type Alias PasswordAuthenticationResponse

```
type PasswordAuthenticationResponse = {
    authenticated: boolean;
    token?: string;
    user: { email: string };
}
```

Defined in controllers/types.ts:18

Index

Properties

- (P) authenticated
- (P) token?
- (P) user

Properties

authenticated

```
authenticated: boolean
```

Defined in controllers/types.ts:19

Optional token

```
token?: string
```

Defined in controllers/types.ts:20

user

```
user: { email: string }
```

Defined in controllers/types.ts:21

Generated using TypeDoc

Type Alias PasswordLoginBody

```
type PasswordLoginBody = {
    email: string;
    password: string;
}
```

Defined in controllers/types.ts:44

Index

Properties

- P email
- P password

Properties

email

```
email: string
```

Defined in controllers/types.ts:45

password

```
password: string
```

Defined in controllers/types.ts:46

Generated using TypeDoc

Module middleware/auth.middleware

Authentication middleware for protected API routes.

Remarks

This middleware is responsible for JWT sent in the `Authorization` header and its validation using the `Bearer <token>` format. If the token is valid, the decoded user information is attached to the request object as `req.authUser`.

If the token is missing, malformed, expired, or invalid, the middleware responds with a `401 Unauthorized` error and does not call `next()`.

This middleware should be applied to any route that requires the user to be authenticated, should be added in routes/ folder.

Utilized src: https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs

Type Aliases

T AuthenticatedRequest

Functions

F authenticateToken

Type Alias AuthenticatedRequest

```
AuthenticatedRequest: Request & { authUser?: { email: string; userId: number }
}
```

Extends Express request type that includes authenticated user data.

Defined in middleware/auth.middleware.ts:33

middleware/auth.middleware / authenticateToken /

Function authenticateToken

```
authenticateToken(
    req: AuthenticatedRequest,
    res: Response,
    next: NextFunction,
): Response<any, Record<string, any>> | undefined
```

Middleware does the verification for the JWT from the `Authorization` header.

Parameters

- req: *AuthenticatedRequest*
- res: *Response*
- next: *NextFunction*

Returns *Response<any, Record<string, any>> | undefined*

Sends a 401 response on failure, otherwise calls `next()`

Remarks

The header must follow the format:

```
Authorization: Bearer <token>
```

If verification succeeds, the decoded payload is attached to `req.authUser`. Otherwise, a `401 Unauthorized` response is sent.

Defined in middleware/auth.middleware.ts:66

Variable verifyEmail Const

```
verifyEmail: ValidationChain[] = ...
```

Email validation middleware for routes that require a valid email address.

Remarks

If validation fails, the error message `"Invalid email format."` will be included in the express-validator error result and should be handled by the global `validate` middleware.

Defined in middleware/auth.validator.ts:15

Variable verifyPassword Const

```
verifyPassword: ValidationChain[] = ...
```

Password validation middleware for routes that require a valid password.

Remarks

If validation fails, the error message `"Password must be at least 8 characters."` will be included in the express-validator error result and should be handled by the global `validate` middleware.

Defined in middleware/auth.validator.ts:32

middleware/validator / validate /

Function validate

```
validate(req: Request, res: Response, next: NextFunction): void
```

Validation result handler for express-validator middleware chains.

Parameters

- req: *Request*
- res: *Response*
- next: *NextFunction*

Returns *void*

Remarks

This middleware should be placed **after** one or more express-validator validation rules. It inspects the accumulated validation errors and throws an `AppError` if any validation rule failed.

NOTE: At this time only the **first** validation error is used for the response.

If no validation errors are present, the request proceeds to the next middleware or controller. Utilized src: https://medium.com/@jyotijingar/express-validator-middleware-for-handling-validation-at-the-request-level-in-express-js-bb263c55e3de

Throws

AppError if validation errors exist

Defined in middleware/validator.ts:25